

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

P. Chola Lakshmi Narasimha

192325021

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P. Chola Lakshmi Narasimha	Reg. No.:	192325021
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

1. User Registration Validation using Regular Expressions

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$$%&!])[A-Za-z\d@#$$%&!]{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'

print("Valid Email" if re.fullmatch(email_pattern, email) else "Invalid Email")
print("Strong Password" if re.fullmatch(password_pattern, password) else "Weak Password")
print("Valid Mobile Number" if re.fullmatch(mobile_pattern, mobile) else "Invalid Mobile Number")
```

2. Deterministic Finite Automata (DFA) Simulator

```
# DFA for strings ending with "ab"

transitions = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}

start='q0'
final={'q2'}

n=int(input("Enter number of strings: "))

for _ in range(n):
    s=input("Enter string: ")
    state=start
    path=[state]
    valid=True

    for ch in s:
        if ch not in ['a','b']:
            valid=False
            break
        state=transitions[state][ch]
        path.append(state)

    print("Transition Path:", " -> ".join(path))
    if valid and state in final:
        print("Accepted")
    else:
        print("Rejected")
```

3. Text Search Engine using Regular Expressions

```
import re

text = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

while True:
    print("\n1.Search Date")
    print("2.Search Phone Number")
    print("3.Search Hashtag")
```

```
print("4.Search Mention")
print("5.Search Prefix")
print("6.Search Suffix")
print("7.Search Word")
print("8.Exit")

ch=input("Enter choice: ")

if ch=="1":
    print(re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text))

elif ch=="2":
    print(re.findall(r'\b[6-9]\d{9}\b', text))

elif ch=="3":
    print(re.findall(r'#\w+', text))

elif ch=="4":
    print(re.findall(r'@\w+', text))

elif ch=="5":
    p=input("Enter prefix: ")
    print(re.findall(r'\b'+re.escape(p)+'\w*', text, re.IGNORECASE))

elif ch=="6":
    s=input("Enter suffix: ")
    print(re.findall(r'\b\w*'+re.escape(s)+'\b', text, re.IGNORECASE))

elif ch=="7":
    w=input("Enter word: ")
    print(re.findall(r'\b'+re.escape(w)+'\b', text, re.IGNORECASE))

elif ch=="8":
    break

else:
    print("Invalid Choice")
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____